CPSC 481
Artificial Intelligence

Chuck Chekuri

cchekuri@fullerton.edu

LinkedIn

Group

Syllabus

Week 6: Heuristic Search Strategies

Moving from "Blind" to "Informed" Search.

The limits of Uninformed Search (Week 5 recap).

How "Hints" change the search efficiency.

Textbook: AIMA 4th Ed. Chapter 3 (Sec 3.5-3.6).

Core Goal: Understanding A* Search and Heuristics.

Recap: Why Uninformed Search Fails

- BFS and Uniform-Cost expand nodes in all directions.
- They lack any sense of "direction" toward the goal.
- Complexity O(b^d) leads to exponential explosion.
- Memory is usually the primary bottleneck for BFS.
- We need domain-specific hints to focus the frontier.

Defining Informed Search

- Also known as "Heuristic Search."
- Uses an Evaluation Function f(n) for node selection.
- f(n) includes a component based on domain knowledge.
- The strategy: Expand nodes that "look" the best.
- Goal: Significantly reduce the number of nodes expanded.

Section 3.5: The Heuristic Function h(n)

- h(n) = estimated cost from node n to the goal.
- It is purely an estimate (a "rule of thumb").
- h(n) = 0 if n is the goal.
- Heuristics are problem-specific.
- Example: Straight-line distance on a map.

Example: Romania and SLD

- Problem: Travel from Arad to Bucharest.
- h_SLD(n): Straight-line distance from city n to Bucharest.
- SLD values are fixed and provided in a table.
- Note: Actual road distance is always >= SLD.
- This "optimism" is key to finding optimal solution.

Greedy Best-First Search (GBFS)

- Strategy: Expand the node closest to the goal.
- Evaluation Function: $f(n) = h(n)$.
- It is "Greedy" because it takes the biggest bite toward the goal.
- Implementation: Priority queue ordered by $h(n)$.
- Ignores the path cost $g(n)$ spent so far.

GBFS Walkthrough: Arad to Bucharest

- Arad has h=366. Neighbors: Sibiu (253), Timisoara (329), Zerind (374).
- Expand Sibiu (lowest h).
- From Sibiu: Fagaras (176), Rimnicu Vilcea (193), others.
- Expand Fagaras (lowest h).
- From Fagaras: Bucharest (0). Found!

Evaluating Greedy Best-First Search

- Completeness: No (can get stuck in loops in tree-search).
- Optimality: No. It finds A solution, not THE solution.
- Time Complexity: O(b^m) in worst case.
- Space Complexity: O(b^m) - keeps all nodes in memory.
- Performance depends entirely on the quality of h(n).

When Greedy Fails: Example

- Consider a path that looks close but is a dead end.
- Greedy will follow the "hint" even if the road is winding.
- It does not "remember" the cost it has already paid.
- Result: Often finds paths that are much longer than optimal.
- We need to balance the hint with the actual cost.

A* Search: The "Gold Standard"

- Pronounced "A-Star."
- Evaluation Function: $f(n) = g(n) + h(n)$.
- $g(n)$: Cost to reach the node (the Past).
- $h(n)$: Estimated cost to the goal (the Future).
- $f(n)$: Estimated total cost of path through node n.

A* Search Logic

- Combines Uniform-Cost Search (g) and Greedy Search (h).
- Uniform-Cost is optimal but slow.
- Greedy is fast but not optimal.
- A* is both fast and optimal (under specific conditions).
- It avoids expanding paths that are already expensive.

A* Walkthrough: Arad to Bucharest

- f(Arad) = 0 + 366 = 366.
- Expand Arad: Sibiu (f=140+253=393), Timisoara (f=118+329=447).
- Expand Sibiu (lowest f).
- From Sibiu: Rimnicu Vilcea (f=140+80+193=413), Fagaras (f=140+99+176=415).
- Rimnicu Vilcea is now the lowest f in the frontier!

A* Optimality: Admissibility

- A heuristic is admissible if it never overestimates.
- $h(n) <= h^*(n)$, where $h^*(n)$ is the true cost to the goal.
- It is "Optimistic" about the remaining cost.
- Admissibility ensures A* won't settle for a bad path.
- SLD is admissible (the shortest distance between two points).

A* Optimality: Consistency

- A stronger condition than admissibility.
- Also known as "Monotonicity."
- $h(n) <= c(n, a, n') + h(n')$ for every action a.
- The estimated cost never drops more than the step cost.
- Consistency ensures f(n) is non-decreasing along any path.

Why Consistency Matters

- If f(n) is non-decreasing, the first time we reach a state, it is via the optimal path.
- This allows us to use Graph-Search without revisiting states.
- Every consistent heuristic is admissible.
- Most natural admissible heuristics are also consistent.
- Visual: The "Contour" lines of search expansion.

Optimality of A* (Proof Sketch)

- Suppose a sub-optimal goal G2 is in the frontier.
- $f(G2) = g(G2) + 0 > C^*$ (optimal cost).
- There must be some node n on the optimal path to G.
- $f(n) = g(n) + h(n) <= C^*$ (because h is admissible).
- Thus, $f(n) < f(G2)$. A* will always pick n before G2.

The Behavior of A*

- A* expands nodes in order of increasing f-value.
- It gradually adds "f-contours" of nodes.
- Nodes with $f(n) < C^*$ will definitely be expanded.
- Nodes with $f(n) = C^*$ may be expanded.
- Nodes with $f(n) > C^*$ will never be expanded.

Efficiency of A*

- A* is "Optimally Efficient."
- No other optimal algorithm is guaranteed to expand fewer nodes.
- However, it can still expand many nodes if h is weak.
- If h(n) = 0, A* becomes Uniform-Cost Search.
- If h(n) is perfect, A* goes straight to the goal.

Drawbacks of A*

- Primarily: Space Complexity.
- Like BFS, it keeps all generated nodes in memory.
- For many problems, A* runs out of memory before time.
- Time complexity is also exponential in the worst case.
- We need variants to handle memory limits.

Memory-Bounded Search Strategies

- Goal: Maintain A*'s properties in limited RAM.
- Iterative Deepening A* (IDA*): Uses f-limit instead of d-limit.
- Recursive Best-First Search (RBFS).
- SMA* (Simplified Memory-bounded A*).
- These algorithms trade time for space.

Recursive Best-First Search (RBFS)

- Mimics standard best-first search.
- Uses linear space (like DFS).
- It remembers the f-value of the best alternative path.
- If current path exceeds the alternative, it backtracks.
- It "unwinds" but remembers the best f-value seen in branches.

Pros and Cons of RBFS

- Pros: Extremely memory efficient. Optimal if h is admissible.
- Cons: Potential for "Regeneration."
- It might re-expand the same nodes multiple times.
- It is "forgetful"—it can't use available memory beyond the current path.
- Often slower than A* if memory is available.

SMA* (Simplified Memory-Bounded A*)

- Uses all available memory to store nodes.
- Acts like A* until memory is full.
- Then, it drops the "worst" leaf node (highest f).
- It stores the dropped node's f-value in its parent.
- This way, it only re-explores if it has to.

Is SMA* the Solution?

- It is complete if the goal is reachable within memory limits.
- It is optimal if the optimal path fits in memory.
- If memory is too small, it may thrash (cycle through nodes).
- It is the most robust memory-bounded variant.
- Performance collapses if memory is severely restricted.

Section 3.6: Developing Heuristics

- The effectiveness of A* depends on the quality of h(n).
- How do we find good heuristics?
- Case Study: The 8-Puzzle.
- Goal: Minimize moves to reach target configuration.
- Two common heuristics: h1 and h2.

8-Puzzle Heuristic 1: Misplaced Tiles

- h1 = number of tiles that are in the wrong position.
- Admissible? Yes. Each misplaced tile must move at least once.
- Simple to calculate.
- Weak estimate: Doesn't consider how far they must move.
- Example: If tile 8 is one move away vs across the board.

8-Puzzle Heuristic 2: Manhattan Distance

- h2 = sum of distances of tiles from their goal positions.
- Distance is |x1-x2| + |y1-y2|.
- Admissible? Yes. A tile must move at least that many times.
- Much more accurate than h1.
- This is the most common heuristic for sliding puzzles.

Heuristic Dominance

- If h2(n) >= h1(n) for all n, then h2 dominates h1.
- Higher values (closer to the true cost) are better.
- A dominant heuristic will expand fewer nodes.
- Example: For depth 14, h1 expands 639 nodes; h2 expands 113.
- Tradeoff: Dominant heuristics might take longer to calculate.

Designing Heuristics: Relaxed Problems

- A problem with fewer restrictions is "relaxed."
- The cost of an optimal solution to a relaxed problem is an admissible heuristic.
- 8-Puzzle Relaxations:
- 1. Tile can move anywhere (Misplaced tiles h1).
- 2. Tile can move to adjacent square even if occupied (Manhattan h2).

Pattern Databases

- Solve a sub-problem and store the results in a table.
- Example: Store the cost to move tiles 1-2-3-4 into place.
- h is then looked up in the database.
- Admissible? Yes, if it represents a true sub-problem cost.
- Can be combined with other patterns for higher accuracy.

Learning Heuristics from Experience

- An agent can solve many versions of a problem.
- Features of states are correlated with "distance to goal."
- Machine learning (e.g., neural networks) can learn h(n).
- Requires many "training" examples of solved paths.
- Common approach in modern game AI (e.g., AlphaZero).

Heuristics for Real-World Problems

- Logistics: Aerial distance as heuristic for shipping.
- Biology: Sequence alignment heuristics.
- Robotics: Visibility graphs and distance to target.
- Challenges: Local optima and high-dimensional spaces.
- Often use "weighted A*" to trade optimality for speed.

The Effective Branching Factor (b*)

- A way to measure the "strength" of a heuristic.
- If A* expands N nodes to find a goal at depth d.
- $N + 1 = 1 + b^* + (b^*)^2 + \ldots + (b^*)^d$.
- A perfect heuristic has $b^* = 1$.
- Manhattan distance often yields b^* near 1.2 for 8-puzzle.

Comparing Informed Strategies

- Greedy: Fast, non-optimal, memory intensive.
- A*: Optimal, memory intensive, efficient.
- IDA*: Optimal, memory efficient, can be slow.
- RBFS: Optimal, memory efficient, regenerates nodes.
- SMA*: Optimal, memory efficient (uses available RAM).

When to Use Which Algorithm?

- Memory is tight? Use IDS or IDA*.
- Optimal solution required? Use A* or SMA*.
- Just need ANY solution fast? Use Greedy.
- Problem is huge? Look for stronger heuristics or patterns.
- State space is a tree? Use Tree-Search variants.

A* in Practice: Navigation

- Used in video game pathfinding (NPC movement).
- Used in GPS navigation (with complex road networks).
- Real-time constraints: Might use "Anytime A*."
- Anytime A* finds a quick solution and improves it over time.
- Visual: Pathfinding in a 2D grid with obstacles.

A* in Practice: Robotics

- Motion planning for robotic arms.
- High-dimensional state spaces (many joints).
- Often uses "Sampling-based" variants.
- Heuristics based on "Clearance" from obstacles.
- Integrating perception and search.

Final Thoughts on Heuristics

- "AI is the study of heuristics."
- Better heuristics = Better performance.
- There is no "perfect" heuristic for all problems.
- Domain knowledge is the most valuable asset in search.
- Next week: Adversarial Search (Games).

Week 6 Review Questions

- 1. What makes a heuristic admissible?
- 2. How does A* differ from Greedy search?
- 3. Why is memory the bottleneck for A*?
- 4. Define consistency in heuristics.
- 5. What is dominance and why do we want it?

Conclusion & Looking Ahead

- Heuristic search is the backbone of classical AI.
- Today: Single-agent, static environments.
- Next: Multi-agent environments (Games).
- Assignment: Compare A* and Greedy on the 8-puzzle.
- Lab: Implementation of A* in Python.